CLOUDHERO / ENGINEERING FIELD GUIDE

Linux server hardening.

Seven areas. Evidence for each.

A reusable operating baseline for the work between security advisories.

SSH / Patches / Kernels / Exposure
Audit logging / Drift / Production verification

2026 edition · Ubuntu 24.04 LTS ARM64 lab
Includes verification steps and recovery considerations.
Companion assessment prompt and lab record supplied separately.

Adoption decision: Linux hardening baseline

Purpose: reduce unauthorized access, unpatched active software, unintended exposure and undetected configuration changes while preserving service availability. This Ubuntu-tested baseline has seven areas and 28 evidence-based controls; application security and business continuity standards remain companion requirements.

Authorize a bounded pilot first. Name the service owner and one accountable person for each area in the operating matrix. Select a representative staging host and one recoverable production canary. Require operator collection on the server, redaction and approved transfer, then AI assessment on a separate workstation without server credentials. Agree the AI evidence-processing boundary before sharing data. The security owner proposes vulnerability deadlines and exceptions; the service owner owns business acceptance and recovery decisions.

Before a production change: complete the policy and change worksheets; inventory identities, software and traffic paths; prove recovery access; identify application backup/recovery evidence; approve the exact change, observation window and rollback trigger. Establish trusted configuration provenance and central logging. Missing required evidence holds the associated gate; an exception needs a named approver, compensating control and expiry and never becomes a PASS.

Rollout: collect and review evidence; test changes and recovery in staging; change one approved canary; verify fresh access, active fixes, allowed/denied paths, log receipt and real workload behavior; observe the agreed thresholds; expand in approved batches. Stop on a failed gate. The service owner signs proceed, hold or rollback at each decision point.

Resource commitment: estimate engineering time for inventory, staging, canary verification and rollout separately. Reserve identity, network, observability and application-owner participation plus recovery capacity. Use pilot measurements to budget each additional host class and the recurring work; do not promise a universal completion time. Onboarding includes maintenance-window effort, and continuous automation still needs failure triage.

Operate and report: review patch failures, exposure and drift daily; review access and log delivery on the approved cadence; verify every material change/reboot; schedule recovery exercises. Report overdue applicable fixes, unsupported hosts, missing detection evidence, unexplained drift and recovery-test completion with denominators, owners, age and due dates. Report unknowns and stale evidence separately from compliant results. Evidence coverage is not a risk score.

Decision record: sponsor, pilot scope, accountable owners, approved policy revision, staffing/maintenance allocation, acceptance thresholds and review date. Approve expansion only after pilot acceptance and unresolved-risk review. The fictional worked change illustrates a complete decision; its numbers are not default policy.

A reusable operating baseline for the work between security advisories. Secure administration, keep supported software running, reduce exposure, and retain evidence that your controls work after a change or reboot.

Control catalogue: LH-1.2, Ubuntu 24.04 LTS ARM64, systemd, OpenSSH, UFW and Linux Audit. The operating principles apply more widely; the commands require adaptation and validation for another distribution, architecture or firewall manager. The companion contains 28 fixed assessment controls, an ownership matrix, a policy worksheet, executable checks and dated lab evidence.

This guide is for people responsible for Linux servers in production. Use it when provisioning a host, reviewing an existing fleet, or responding to a CVE. It is a baseline, not a complete application security review or a compliance certification. Identity governance, backup recovery, encryption, secrets, mandatory access control, supply-chain security and workload isolation need their own policies and tests.

Workflow revision: toolkit LH-1.2.1, 5 November 2026. Collection scripts and control criteria are unchanged; the dated VM evidence retains its original scope.

Adopt the baseline

Start with the one-page adoption brief: pilot scope, one accountable owner per area, staffing, rollout gates and recurring reporting. Complete the policy and change worksheets before production work. The fictional completed change shows thresholds, ownership, evidence and a hold/rollback decision without inventing a production receipt.

Start with an assessment

Required workflow: collect on the Linux server, transfer a redacted copy, then run the assessment prompt on a different machine. An authorized operator runs the scripts. Claude or Codex analyses the supplied evidence on your workstation. The prompt must not download or execute collectors, connect to the server, request SSH/sudo access or inspect the workstation hosting the analysis.

The scripts collect facts and perform specific deterministic checks. The prompt interprets evidence against your policy and evaluates the 28 controls. The current scripts do not independently produce a complete 28-control assessment. Neither collection nor AI assessment applies hardening fixes; remediation is a separately approved task. The modifying `lab-*` fixtures are for disposable test VMs only.

Step 1: collect on the approved Linux server

An authorized operator obtains the toolkit from the GitHub release assets, extracts it and reviews `assess.sh` and `collect-report.sh`. On the intended Ubuntu target, from a private working directory containing the reviewed scripts, run:

```
# SERVER: run this yourself on the approved Linux target.
bash collect-report.sh ./assess.sh
```

The wrapper requests the operator's approved sudo access when needed; the AI does not receive those credentials. It creates a new `0700` directory and `0600` report at `linux-assessment.XXXXXXXX/assessment.txt`, prints the actual path and preserves the collector's exit status, including failed-collection evidence. It never reuses an existing file. The collector reads configuration without installing packages, refreshing metadata, changing settings, restarting services or rebooting. Normal access can itself create access logs.

Step 2: redact and transfer the evidence

Keep the original report private on the server. Review a separate copy and remove secrets, personal data and infrastructure details your policy does not permit sharing. Use a consistent target alias in the report and policy; preserve collection timestamps, command exit statuses and the facts needed for each check. Record redactions that limit interpretation. A missing or stale observation cannot create a PASS.

Transfer only the approved, redacted copy to a different machine using your established secure transfer method. Include the completed `policy-template.json`, target alias, workload role, environment and any approved evidence for network paths, central logging or recovery. Do not copy SSH private keys, passwords, tokens or unrestricted logs. Approve the AI service, processing location, retention and evidence classification before sharing: a cloud model processes supplied evidence even when you use it from your own workstation.

Step 3: run the prompt on your analysis workstation

On the separate machine, open your approved Claude or Codex environment. Supply the full `linux-assessment-prompt.txt`, redacted `assessment.txt`, completed policy and target context. Give the model access only to the approved evidence and assessment output area. It needs no server credentials, production shell or sudo access. A prompt is not an OS permission boundary; enforce that separation in the analysis environment.

Explicitly include this handoff instruction alongside the full assessment prompt:

```
Assess only the server described in the attached redacted evidence.
Required mode: operator collection on the server; AI analysis here.
This is a separate analysis workstation, not the assessment target.
Do not download or execute scripts, connect to the server,
request SSH/sudo access, or inspect this workstation.
Use the supplied policy and collection timestamps.
If evidence is missing, request it from the operator; never guess PASS.
Produce the 28-control report and proposed fixes only. Do not remediate.
```

If no report is supplied, the prompt asks the operator to collect, redact and transfer it first. It must not switch to live collection. For missing observations, it can propose exact read commands for the authorized operator to review and run on the server; the operator returns another redacted receipt. Only the server described in the evidence is assessed. The workstation's OS and account are irrelevant to the target's findings.

Step 4: review the report before remediation

Require a Markdown assessment and `report.json`. On the analysis workstation, an operator can validate the report structure using the local toolkit:

```
# WORKSTATION: validates the report; does not inspect the server.
python3 check-report.py --catalog controls.json report.json
```

The validator checks IDs and required evidence fields, not whether conclusions are true. Review every finding, remaining evidence gap and proposed change before arranging approved remediation. After maintenance, repeat server collection and workstation analysis with fresh evidence. The completed report describes the supplied collection times, not continuous monitoring or a fresh server scan.

Use all 28 IDs from the fixed LH-1.2 catalogue. Apply an evidence-backed, approved N/A only when the applicability condition is met. Otherwise, a proven unmet criterion is FAIL even if other observations are missing; a failed required collection is ERROR; missing required evidence or policy is MANUAL. With complete required evidence and no failed criterion, an explicit policy warning threshold produces WARN; all criteria satisfied without that warning produces PASS. Do not use WARN to hide missing evidence or an exception to convert FAIL into PASS. Report status counts and evidence coverage; neither is a security score. Additional findings do not change the denominator. Compare runs only when catalogue version and scope are the same.

Before changing a server

Record its owner, workload, distribution, release, architecture, kernel flavour, service ports and maintenance window. Confirm a working console or out-of-band recovery path. Keep the existing admin session open, test a second key-authenticated session and verify approved privilege escalation before editing SSH or the firewall.

Back up the exact configuration files you will change, including ownership and permissions. Take a recoverable infrastructure snapshot where appropriate and verify application backup recovery separately: a disk snapshot alone does not prove a database is recoverable. Define health checks, rollback triggers and who decides whether to proceed. Start with a canary host and drain traffic when required.

All executable remediation examples below target **Ubuntu 24.04 LTS with systemd, OpenSSH, UFW and auditd**. They are individual change examples, not a script to paste into production. The lab uses ARM64; an x86 host or another distribution needs its own validation. Do not enable UFW on a host already managed by firewalld, a native nftables policy or a container networking controller without planning ownership.

Prerequisites and change boundaries

Read commands can run during an authorized assessment. Package installation, file creation, reloads, test-event generation and reboots belong to a separately approved maintenance task. The downloadable policy template leaves organization-specific values blank deliberately: a missing deadline, owner or acceptance threshold is an unresolved decision.

On the disposable Ubuntu lab, these packages supplied the examples. For an existing server, inspect what is installed and review the transaction before adding anything. Installation can start services. Keep AppArmor/SELinux enforcing and investigate denials rather than turning protection off to make a test pass.

```
# Approved staging/maintenance only:
sudo apt-get -o APT::Update::Error-Mode=any update
sudo apt-get -s install openssh-server ufw auditd unattended-upgrades needrestart
sudo apt-get install openssh-server ufw auditd unattended-upgrades needrestart
sudo install -d -m 0755 /etc/ssh/sshd_config.d
sudo install -d -m 0755 /etc/systemd/journald.conf.d
sudo install -d -m 0755 /etc/audit/rules.d
```

Expected: all repositories refresh successfully, the proposed packages match the chosen stack, and prerequisite directories exist. **Stop:** missing entitlement, unsigned/untrusted repositories, unresolved package errors, a competing firewall manager or no recovery access. Preserve exact pre-change files and metadata. Record the change ID and baseline revision before applying the examples.

1. SSH: preserve access while removing weak paths

Use named identities with short-lived certificates or well-managed keys. Limit administration to your VPN, bastion or approved management networks. Remove direct root login after proving that the named administrator can perform required tasks. Review unused accounts, keys, certificate authorities, authorized-key restrictions and revocation procedures with the identity owner.

Inspect the effective configuration, not just a line in a file. OpenSSH generally uses the first obtained value; Include order and Match blocks matter. A later file named `99-hardening.conf` may not override an earlier setting. Evaluate each relevant user and source address, including root and any special-purpose accounts. OpenSSH configuration reference.

```
sudo /usr/sbin/sshd -t
sudo /usr/sbin/sshd -T
# Replace the values with the real connecting user, client host and source IP.
sudo /usr/sbin/sshd -T \
  -C user=admin,host=admin-client,addr=192.0.2.10
```

Here `host` means the connecting client's hostname. Include `laddr` and `lport` when your Match rules depend on the destination address or port. Do not assume the example account or documentation address exists on your server.

For a **key-only server that does not use PAM keyboard-interactive MFA**, a reviewed drop-in can contain the following. Preserve `UsePAM` and your distribution's authentication/session controls. If you use MFA, assess the complete `AuthenticationMethods` chain instead of disabling its mechanism.

```
# /etc/ssh/sshd_config.d/00-cloudhero.conf
PermitRootLogin no
PubkeyAuthentication yes
PasswordAuthentication no
KbdInteractiveAuthentication no
X11Forwarding no
AllowAgentForwarding no
AllowTcpForwarding no
LogLevel VERBOSE
```

Disable forwarding only when the server's role permits it; bastions and developer workflows may depend on it. Do not copy arbitrary cipher lists that drift away from supported vendor defaults. Changing the SSH port reduces noise but does not replace authentication and network restrictions.

Apply and verify: validate syntax, inspect all relevant effective contexts, then reload the service. On the tested Ubuntu image the service is `ssh`; other distributions may use `sshd`. A change to listening addresses or ports can also involve socket activation and needs its own procedure. This example changes authentication only. Ubuntu OpenSSH guide.

```
sudo /usr/sbin/sshd -t && sudo systemctl reload ssh
```

Open a new connection from an approved source. Replace the example identity and host with the actual target; verify the host key through your established trust channel.

```
ssh -o ControlMaster=no -o ControlPath=none -o BatchMode=yes \
  admin@server.example 'id -un'
```

Expected: the named identity and exit 0 prove a fresh key login. Verify privilege separately according to the identity's approved role. For an interactive administrator, open a fresh terminal with `ssh -t -o ControlMaster=no -o ControlPath=none admin@server.example`, run `sudo -l`, complete required authentication, and perform the specific approved task from the runbook. For an automation identity already authorized for noninteractive sudo, use `sudo -n` with its exact approved command and arguments. A password prompt is valid policy for a human administrator; failure of `sudo -n true` does not by itself mean access is broken. Do not expand privileges to make a generic probe pass. SSH-03 needs both the fresh-login receipt and successful role-appropriate task evidence. Ubuntu sudo reference.

For this key-only profile, effective output includes `permitrootlogin no`, `passwordauthentication no`, `kbdinteractiveauthentication no` and `pubkeyauthentication yes`. A context that permits an unintended authentication path fails SSH-01 even if the main configuration file looks correct. Verify key removal/certificate expiry and offboarding separately; named accounts alone do not prove lifecycle control.

Stop and recover: keep the original session until the new one succeeds. On failure, use it or the console to restore the exact previous SSH files, validate and reload, then test a new session again. Do not close the last working session. Schedule and record an independent console recovery and access-revocation exercise for SSH-04.

Lab troubleshooting: after an OpenSSH package update, the socket was listening while the service was inactive; `sshd -t` reported a missing `/run/sshd`. A fresh authorized connection activated the service and restored its runtime directory. Inspect `ssh.service` and `ssh.socket`; do not interpret this error as a clean syntax result or blindly create runtime directories to hide it.

2. Patch management: define how fixes reach production

Inventory operating-system packages, third-party repositories, agents, language runtimes and applications outside the package manager. Assign patch deadlines based on exploit activity, exposure and business impact. Document the owner of exceptions, the compensating control and an expiry date. A configured update timer needs evidence of recent successful runs.

For Ubuntu, inspect holds, package consistency and the proposed transaction before an approved update. The simulation uses existing metadata; stale indexes can hide available fixes. Refreshing metadata changes local package-manager state and belongs in an authorized maintenance task, not the observational assessment.

```
apt-mark showhold
sudo dpkg --audit
sudo apt-get -s upgrade
systemctl list-timers apt-daily.timer apt-daily-upgrade.timer
# Authorized maintenance only:
sudo apt-get -o APT::Update::Error-Mode=any update
sudo apt-get -s upgrade
```

Use your tested patch pipeline or unattended security updates. On Ubuntu, `unattended-upgrades` selects updates according to its allowed origins; third-party repositories are not automatically covered. Inspect the effective settings and run a dry run in staging. That dry run may write logs, so it is not part of the read-only prompt. Ubuntu automatic updates.

```
# /etc/apt/apt.conf.d/20auto-upgrades
APT::Periodic::Update-Package-Lists "1";
APT::Periodic::Unattended-Upgrade "1";

# /etc/apt/apt.conf.d/52cloudhero-reboot
Unattended-Upgrade::Automatic-Reboot "false";
```

```
# In staging or an approved maintenance window:
sudo unattended-upgrade --dry-run --debug
sudo apt-get upgrade
sudo needrestart -r l
```

`apt-get upgrade` applies all eligible upgrades from configured repositories, not just security updates. It can keep packages back when they require new dependencies. In this lab, that left the kernel

meta-packages on the old revision. Review the following simulation, then explicitly approve its additional packages before running the second command. This option permits new packages without removing installed ones; do not turn a broad `full-upgrade` into an automatic removal decision. Ubuntu APT command reference.

```
sudo apt-get -s --with-new-pkgs upgrade
# Only after reviewing the proposed new dependencies:
sudo apt-get --with-new-pkgs upgrade
```

Package scripts can restart services even when automatic reboots are disabled. `needrestart -r l` lists restart needs; arrange the actual restarts through your change process. In the lab it initially reported the running kernel as up to date while newer kernel packages were still kept back. Pair restart checks with repository candidate and meta-package checks.

Verify: capture the completed transaction, investigate failures and holds, compare installed vendor revisions with the relevant advisory, and confirm affected services are using the updated code. Package rollback is not always available or safe; define a tested image, snapshot or application recovery path before maintenance.

Expected: a successful refresh for every required source, no unexplained holds or kept-back packages, no incomplete dpkg transaction, and a completed update/pipeline receipt. A timer's enabled state is insufficient: inspect the actual last successful run, intended security origins and failure notifications. The strict refresh option makes transient retrieval errors fail the operation. An unreachable mirror means "inventory incomplete", not "zero updates needed".

Use `apt-cache policy` to inspect installed and candidate versions for the affected package and the actual kernel meta-package. The lab uses `linux-image-virtual`; an AWS, Azure, generic or HWE host can use a different supported track. Do not switch tracks merely to match an example. For a CVE, first establish the exact vendor release/package applicability, then compare complete Debian revisions with `dpkg --compare-versions`. A true comparison still needs active-process/kernel verification.

Stop and recover: hold rollout on an unexpected removal, unresolved dependency, failed refresh, unsupported source or failed workload gate. Do not force package downgrades or delete package-manager lock files. Restore the tested image/application state if required, reopen the vulnerability finding and apply a time-limited mitigation. Record failed update alerts and exception expiry with PATCH-03/04.

3. Kernel lifecycle: installed is not running

Track three states separately: the kernel currently running, kernel packages installed on disk, and the kernel the bootloader will select next. Keep the distribution's appropriate kernel meta-package so future security kernels continue to arrive. Verify the supported track for the exact OS release, hardware/cloud flavour and subscription coverage.

```
uname -r
dpkg-query -W 'linux-image-*'
df -h /boot /
test ! -f /run/reboot-required || cat /run/reboot-required
test ! -f /run/reboot-required.pkgs || cat /run/reboot-required.pkgs
sudo needrestart -r l
```

An absent reboot marker does not prove the running kernel contains a CVE fix. Do not choose the next-boot kernel by sorting filenames alone; inspect the actual bootloader/default entry and any saved selections. Keep a known-good fallback and verify console access. Avoid removing the running kernel or the only recovery kernel to free disk space.

Livepatch can reduce exposure for covered vulnerabilities, but coverage depends on the kernel, service state and specific fix. It does not eliminate kernel lifecycle management or every reboot. Verify livepatch status and the advisory rather than inferring protection from an installed client. Canonical's reboot guidance.

Apply and verify: drain the canary, perform an approved reboot, then compare `uname -r` and package ownership with the intended supported kernel. Check boot errors, drivers, storage mounts, networking and application behavior before returning traffic. If the kernel fails, use the console to select the tested fallback, then investigate; rolling back may reintroduce the vulnerability.

For the tested GRUB image, inspect the default and saved selections together with the generated menu entries. UEFI/systemd-boot and cloud image-replacement workflows need their own supported equivalent. A `/boot/vmlinuz` symlink alone does not prove what a bootloader will choose.

```
cat /proc/sys/kernel/random/boot_id
dpkg-query -W 'linux-image-*'
sudo grep -nE '^[[:space:]]*GRUB_(DEFAULT|SAVEDEFAULT)=' /etc/default/grub
sudo grub-editenv list
sudo grep -nE '^[[:space:]]*(submenu|menuentry|linux|linuxefi)[[:space:]]' /boot/grub/grub.cfg
```

This excerpt preserves indented titles and submenu declarations; it is a navigation aid, not a boot-selection parser. If `/etc/default/grub.d` exists, inspect its applicable `.cfg` overrides too. Source settings may not yet have been regenerated into the active `grub.cfg`. Read the generated selection logic as text without sourcing or executing it: confirm whether `next_entry` takes precedence, whether the default uses `saved_entry`, or whether a fixed index/title/ID is selected. Follow the entire submenu path to the associated `linux`/`linuxefi` image. For example, `Advanced options for Ubuntu>Ubuntu, with Linux 6.8.0-142-generic` identifies a nested entry, not the first kernel printed by grep. Retain that mapping with the boot environment evidence. Custom boot logic, another active bootloader or an unresolved path leaves KERN-02 MANUAL. GNU GRUB selection reference.

Expected: the reviewed next-boot entry names the intended image; after reboot the boot ID changes and `uname -r` names that image. The old kernel stays available until recovery requirements are satisfied. KERN-02 remains open if the boot choice cannot be established. KERN-04 requires an actual recovery exercise, not merely an old image on disk. The companion lab booted the retained `6.8.0-142-generic` entry once, checked access/services/auditing, then returned to default `6.8.0-147-generic`. This proves a

planned fallback boot on that VM; it does not prove recovery from every unbootable system or meet your application's recovery objective.

Stop and recover: do not reboot a second node until the canary passes access, storage, driver, networking and workload checks. If fallback is needed, record the reverted kernel, reopen any affected CVEs, keep compensating exposure controls and establish a new remediation deadline.

4. Firewall exposure: test the path packets take

Build a small service inventory: listener, protocol, address family, expected source, business purpose and owner. Check the host, cloud security groups/firewalls, load balancers, VPN routes and container networking together. A listener on `0.0.0.0` or `::` is not proof of public reachability, and a quiet local port scan is not proof that every external path is closed.

Worked exposure inventory

- **Administration:** TCP 22, IPv4 and IPv6 if deployed, from named VPN/bastion ranges only; platform owner. Required proof: approved-source connection succeeds; an untrusted-source connection fails against the same live SSH listener.
- **Public web entry:** TCP 443 only where this host is the intended public endpoint; service owner. If a load balancer is the sole entry point, use its approved source paths instead of allowing the whole internet at the host.
- **Application backend:** TCP 8080 bound to loopback or permitted only from approved proxies; application owner. Prove it is unreachable from the untrusted test source while the backend is running.
- **Dependencies:** outbound DNS, NTP, package mirrors, central logging and workload endpoints; network/service owner. Include actual UDP dependencies and record return-traffic handling.

Download the exposure CSV and fill in real sources, destinations, interfaces, cloud rules and test vantage points before changing a rule. Documentation addresses below are never an approved production source list.

```
sudo ss -lntu
sudo ufw status verbose
sudo nft list ruleset
```

Read the native rules to understand the resulting policy; do not edit UFW-managed nftables/iptables chains directly. On a new UFW-managed host, allow the actual administration source and required application traffic **before** enabling default-deny ingress. These are documentation ranges and examples; replace them, verify your source as the server sees it, and preserve console recovery. Add the correct IPv6 management prefix when that path is used. Ubuntu firewall documentation.

```
# Example values only. Adapt to the approved service inventory.
sudo ufw allow from 192.0.2.10/32 to any port 22 proto tcp
sudo ufw allow from 2001:db8:100::10/128 to any port 22 proto tcp
sudo ufw allow 443/tcp
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw enable
sudo ufw status verbose
```

The egress allowance is a starting policy, not egress control. If you restrict outbound traffic, inventory DNS, NTP, package mirrors, telemetry and application dependencies first. Do not block required ICMPv6 blindly; neighbour discovery and path MTU behavior matter.

Docker-published ports can be routed before the normal UFW input policy. Review published mappings, forwarding chains and the backend-specific Docker guidance; do not disable Docker's firewall integration as a shortcut. Docker firewall behavior.

Verify: from authorized management and untrusted test sources, check only approved target ports over both IPv4 and IPv6. Prove that allowed traffic succeeds and disallowed traffic fails while a known listener exists. Include UDP where used. Record source, destination, port, family and result. If no external vantage point is available, mark external exposure unverified. Restore the exact prior rules from the console if the change interrupts required traffic.

Expected: every listener has an inventory row; every deployed address family has explicit policy and probe evidence. Confirm UFW's IPv6 configuration and existing rules. Changing the default policy does not remove a pre-existing broad allow rule. A firewall timeout without a known healthy listener and positive control is inconclusive. For UDP, require an application-level request/response or receiver evidence; an absent response alone does not establish filtering.

Stop and recover: before changing management access, prove an independent console/recovery route and retain the exact host and cloud policy. Restore through that route if access or dependencies fail; verify both allowed and denied paths after restoration. The lab's temporary application allowance is removed and original firewall files are compared byte-for-byte. NET-02/03 remain MANUAL for cloud and internet paths until those environments provide evidence.

5. Audit logging: prove an event reaches a person

System journals describe service behavior. Linux Audit records configured security events. Neither a running daemon nor a directory full of logs proves that the events you need are being retained and reviewed. Set retention according to investigation needs, privacy requirements, disk capacity and central storage policy.

For a small host, the following is a starting journal policy; size it for the workload. Persistent storage keeps local journals across boots, subject to limits and available disk. systemd journal configuration.

```
# /etc/systemd/journald.conf.d/60-cloudhero.conf
[Journal]
Storage=persistent
SystemMaxUse=256M
SystemKeepFree=512M
```

```
sudo systemctl restart systemd-journald
sudo journalctl --flush
sudo journalctl --list-boots
sudo journalctl --disk-usage
timedatectl show -p NTPSynchronized
```

Create targeted audit rules for changes to SSH and privilege configuration. These rules are a narrow starting point, not a full audit policy. Ensure the referenced paths exist. The lab exercised the 64-bit rules on ARM64; systems that run a 32-bit ABI may need additional architecture-specific rules validated on that system. Linux Audit rules.

```
# /etc/audit/rules.d/50-cloudhero.rules
-a always,exit -F arch=b64 -F path=/etc/ssh/sshd_config -F perm=wa -k ssh_config
-a always,exit -F arch=b64 -F dir=/etc/ssh/sshd_config.d -F perm=wa -k ssh_config
-a always,exit -F arch=b64 -F path=/etc/sudoers -F perm=wa -k privilege_config
-a always,exit -F arch=b64 -F dir=/etc/sudoers.d -F perm=wa -k privilege_config
```

```
# Approved configuration change only:
sudo augenrules --load
sudo auditctl -l
sudo auditctl -s
```

`augenrules` assembles persistent rule files and loads the resulting policy. If auditing is immutable (`enabled 2`), a reboot may be needed to change it; do not disable protection to force a reload. Watch the backlog and `lost` counter under representative load. Define disk-full and forwarding-failure behavior deliberately rather than copying a setting that could halt a production workload. augenrules reference.

Audit retention is separate from journal retention

The journal's `SystemMaxUse` does not limit `/var/log/audit/audit.log`. Inspect the effective auditd configuration separately:

```
sudo grep -Ev '^[[:space:]]*(#|$)' /etc/audit/auditd.conf
sudo auditctl -s
df -h /var/log /var/log/audit
```

Expected: writing is enabled; rotation or archival matches the retention policy; low-space, full-disk and write-error actions have named responders. As an illustrative capacity calculation, 8 files at 64 MiB is roughly a 512 MiB local rotation budget, not a guarantee of a number of days. Measure event volume and

allow headroom. Keeping every log indefinitely can exhaust disk; suspending logging can leave a running but blind service. Record these availability/detection tradeoffs explicitly. Ubuntu auditd configuration.

Verify rotation in staging with an approved rotate operation, then prove a new event reaches the active file and an earlier event remains available. Exercise low-space/queue alert logic on a disposable fixture; never fill production storage to test an alert. Capture audit `lost` counters before and after a representative workload and investigate any increase.

A concrete central-delivery test

The companion rsyslog example uses a trusted CA, expected receiver certificate identity, client certificate, bounded disk-assisted queue and retry policy. Provision the organization's certificates and validate `rsyslogd -N1` before a reload/restart. Keep the distribution's confinement policy enabled; grant only the required paths through the reviewed policy. The lab had to move its fixture into permitted paths rather than disable AppArmor. rsyslog TLS client guidance.

A syslog marker does not demonstrate Linux Audit ingestion. Include an explicit approved audit input or supported audit collector, with permissions and confinement validated, and test that stream separately. The downloadable examples name the certificate, input and queue assumptions; use one ingestion route per stream to avoid duplicates.

```
# Approved delivery exercise only; record this exact ID and UTC timestamp.
MARKER="linux-baseline-$(date -u +%Y%m%dT%H%M%SZ)"
logger -t linux-baseline "$MARKER"
sudo auditctl -m "$MARKER-audit"
```

Expected receiver receipt: marker ID, host alias, source-event UTC time, receiver UTC time, stream and measured delivery delay. Search the central receiver for both IDs. Then trigger the approved detection rule and retain its ticket/notification plus human acknowledgement. Match the delivery and acknowledgement deadlines from the policy worksheet. Missing receiver access leaves AUD-03 MANUAL; missing human acknowledgement leaves AUD-04 MANUAL.

In staging, reject a receiver with the wrong certificate identity, simulate a bounded receiver outage, recover it and reconcile every expected sequence ID at the sink. Record the expected count, unique receipts, missing IDs and duplicates; a final marker alone cannot prove delivery of earlier messages. TCP/TLS and a queue do not guarantee exactly-once delivery; use an acknowledged protocol or an application-level reconciliation requirement where that is necessary. The revised lab observed all 300 messages sent after connection refusal, with no missing IDs or duplicates. That proves the tested queue-recovery sequence, not preservation of ambiguous in-flight messages during an abrupt disconnect. rsyslog forwarding and queue guidance.

For queue monitoring, use the named action's `impstats` queue size, disk-spool use, discard counters and forwarding diagnostics. Agree thresholds, measurement freshness, alert destination and response deadline. A disconnected exporter or missing metrics must alert rather than report an empty queue. The lab triggered a real queue-size threshold and wrote a receipt to an isolated local alert sink while keeping the filesystem below its capacity limit. Production notification routing and human acknowledgement remain AUD-04 evidence. AppArmor remained enforcing; restricted optional resource metrics did not substitute for

the observed queue counter. Disk-full behavior and crash durability are separate tests, not implied by a queue-pressure exercise. rsyslog statistics.

Verify: in staging, make a harmless controlled change to a watched configuration file, retrieve its event by key, restore the exact original file, and verify its checksum. Separately send an approved unique marker through the central pipeline and record its receipt and alert routing. After a reboot, prove that earlier journals remain and audit rules reload. Use authenticated transport, restricted central access and retention controls; root on a host can tamper with its local evidence. The lab verifies local capture and persistence; central delivery and human response require your production environment.

6. Configuration drift: keep a trusted desired state

Put the intended package sources, SSH policy, firewall rules, logging policy, service configuration and kernel track in a reviewed configuration repository. Record the approved revision with each deployment. Use your existing configuration-management system to compare expected and observed state, including extra files, removed files, permissions, enabled services and package inventory.

A reviewed inventory must cover more than file bytes. The companion `baseline.py` records path names, file types, owners, permissions, file hashes and symlink targets. Directory scope detects unexpected drop-ins and removed files. The starter scope includes the complete `/etc/apt`, journald drop-in, UFW, sudo and rsyslog configuration locations, alongside SSH, Audit and local systemd units. This detects a new late APT or journald override as well as changes to before/after firewall rules. It hashes regular file bytes locally; it does not print file contents. Treat the manifest and paths as sensitive metadata, especially where configuration contains credentials.

Review include paths and overrides for the actual host: vendor directories, runtime overrides and symlink targets can live outside this starter scope. Symlinks are recorded without traversing their targets; add approved target paths separately when needed. Missing mandatory roots fail collection; explicitly document any legitimate scope adaptation instead of silently skipping it. Package and runtime state require separate comparisons. Authenticate the path list and manifest using an independently controlled repository/artifact digest before trusting a local comparison. Ubuntu configuration ordering.

```
# Review baseline.py and baseline-paths.json first. Every listed root must exist.
# Approved creation only, from a quiescent, reviewed configuration:
sudo install -d -m 0700 /var/lib/cloudhero-baseline
sudo python3 baseline.py create --paths baseline-paths.json \
  --manifest /var/lib/cloudhero-baseline/files.json
# Subsequent observational comparison against the independently approved scope:
sudo python3 baseline.py check --paths baseline-paths.json \
  --manifest /var/lib/cloudhero-baseline/files.json
```

Expected: exit 0 means the scoped inventory matches; exit 1 means drift; exit 2 means a collection/input error. A missing input during creation must leave the previous manifest unchanged. Creation writes a private temporary file and replaces the manifest only after the entire inventory succeeds. A failed collection must never replace an approved baseline with a partial one.

For integration with `assess.sh`, an authorized operator can install the reviewed tool at `/usr/local/lib/cloudhero/baseline.py` and the approved scope at `/etc/cloudhero/baseline-paths.json`, owned by root and not writable by the assessment account. Keep the approved manifest at `/var/lib/cloudhero-baseline/files.json`. The collector performs a comparison only when these exist; it never creates or promotes a baseline. Keep the authoritative copies and digests outside the target host.

Verify: test content change, an unexpected SSH drop-in, file removal, changed mode/owner and symlink substitution in staging. Restore the original state and obtain a clean comparison without regenerating the manifest. The companion regression suite includes these cases and proves that missing or unsupported inputs cannot replace the approved manifest. It is a quiescent inventory tool, not a defence against an attacker concurrently controlling root; local root can also rewrite the tool and manifest.

Compare these additional inventories with their reviewed revision: installed package versions and holds; enabled service units and overrides; effective SSH contexts; active firewall rules; kernel track; central logging destinations. Normalize only fields known to be volatile, not meaningful differences. Use the configuration manager's safe observation mode after reviewing any plugins it can execute. A filesystem MATCH alone cannot pass DRIFT-03.

Stop and recover: never refresh the baseline simply to remove an alert. Triage the change against an authorized deployment; investigate unexplained security-sensitive drift before overwriting potential evidence. Restore from the approved source if appropriate, repeat behavior tests, and promote a new baseline only through a reviewed change. Track exception owner, compensating control, approver and expiry.

7. Production verification: close the change with evidence

Define acceptance criteria before maintenance. Capture the same checks before the change, after each meaningful step and after reboot. Use a canary and stop the rollout if access, service health or detection degrades.

```
uname -r
systemctl --failed --no-pager
sudo dpkg --audit
sudo ss -lntu
sudo /usr/sbin/sshd -t
sudo auditctl -s
sudo journalctl --list-boots
```

These commands are an operating-system checkpoint. Complete the following with workload-specific evidence:

- A fresh authorized admin login and required privilege escalation succeed; the recovery path is available.
- The intended kernel and updated services are active; package transactions are complete and restart requirements are resolved or tracked.

- Approved ingress succeeds and denied ingress fails from the intended source networks over every deployed address family. Cloud and container paths are included.
- A representative user transaction, background job and dependency call succeed. Latency, error rate and saturation remain within your acceptance thresholds for an agreed observation window.
- Security events reach the central destination and the responsible team can receive the alert. No unexplained event loss or disk pressure is present.
- Configuration matches the approved revision; exceptions have owners and expiry dates. Recovery and rollback evidence is attached to the change.

If any item lacks evidence, keep it open. A health endpoint returning HTTP 200 does not prove checkout, authentication, queue processing or data integrity. Record what was actually tested, by whom, when and from where. Return a canary to service only after its acceptance criteria pass, then expand the rollout in stages.

A completed decision gate

Use the change worksheet to specify the exact transaction, expected result, maximum error/latency change, observation period and minimum healthy capacity before maintenance. These values come from the service's objectives; a generic hardening article cannot choose them for your application.

In the companion's synthetic application, release A passes a representative transaction. Release B returns HTTP 200 from its health endpoint but produces the wrong transaction result. The verifier marks the gate failed, holds advancement, restores release A and proves the expected transaction again. The recorded result demonstrates rollback behavior; a production service must supply its own real transaction, dependency, background-job and recovery evidence.

Expected: VERIFY-01 has complete criteria and owners; VERIFY-02 has successful OS/access checks; VERIFY-03 has real workload results within thresholds; VERIFY-04 has an explicit proceed/hold/rollback decision. A test command that exits successfully but returns the wrong semantic result fails the gate. Collect failed-unit names and `dpkg --audit` output, not just their command exit statuses.

Stop and recover: abort on loss of management access, unexpected public exposure, missing logs, failed representative work or breached acceptance thresholds. Do not advance another batch while the failed gate is unresolved. Restore the tested state, repeat the same acceptance tests and record whether the vulnerability is reintroduced. A rollback that restores uptime but loses required data is not a successful recovery.

Use this baseline in an individual CVE article

Keep the evergreen controls here. A CVE article should add the facts that change: affected distributions/packages, vendor fixed revisions, exploit prerequisites, current exploitation information, temporary mitigations, required restarts and a verification date. Link directly to the relevant control instead of repeating the whole hardening guide.

Use this compact response record for each affected server:

- **Applicability:** exact OS release, architecture, package/flavour, exposure and feature prerequisites; vendor advisory URL and checked-at time.
- **Fix state:** installed vendor revision, fixed vendor revision, active process/kernel revision, and any livepatch coverage. Vendor backports mean upstream version strings alone can mislead.
- **Action:** remediation owner, deadline, canary, maintenance requirements, mitigation expiry and tested recovery path.
- **Proof:** post-change version evidence, required reboot/restart evidence, application checks, exposure checks and central log receipt.

Use explicit outcomes: affected; not affected with vendor evidence; fixed on disk but restart pending; fixed and active; or unknown. Hardening can reduce exploit paths, but it is not proof that a vulnerability is fixed. Consult the current Ubuntu security notices or the applicable distribution vendor's advisory for each CVE.

A repeatable operating cadence

At provisioning, establish ownership, support coverage, access, logging, approved configuration and recovery. For each advisory, assess applicability and prioritize the specific fix. During maintenance, stage updates, verify the canary, reboot or restart when required and record acceptance evidence. On your agreed recurring schedule, review stale exceptions, patch failures, new exposure, log delivery and configuration drift.

Turn the guide into an owned service

The companion operating matrix assigns each area an accountable role, review trigger and evidence obligation. Suggested starting cadences are daily patch/exposure/drift monitoring, monthly access and log-delivery review, and acceptance checks for every change or reboot. They are planning examples; the organization's policy can require a different interval.

The security and service owners set patch deadlines using active exploitation, reachable attack paths, vendor severity and business impact. Record numeric deadlines in the policy worksheet; blank values remain MANUAL. For an urgent exploited exposure, identify the incident owner immediately, assess containment and establish an explicit remediation deadline rather than waiting silently for the routine window.

Track decision-useful measures with clear denominators: overdue applicable patches / all applicable patches; unsupported hosts / managed hosts; hosts with current central-delivery evidence / hosts requiring it; unresolved security-sensitive drift / assessed hosts; successful recovery exercises / scheduled exercises. Show stale and unknown evidence separately. Neither a green host count nor assessment coverage measures residual risk by itself.

Every exception needs an affected control ID, scope, reason, owner, compensating control, approver, expiry and revalidation date. Expired exceptions reopen the finding. Before adopting this seven-area baseline, link the organization's separate standards for least privilege, AppArmor/SELinux, secrets, backups, encryption, supply-chain verification and workload isolation. Those domains are prerequisites or companion reviews, not implicitly certified by this guide.

For other distributions, preserve these outcomes but use the vendor's supported package manager, firewall manager, service names, audit architecture and kernel lifecycle. RHEL has advisory-aware security update workflows; Amazon Linux, Debian and SUSE have their own support and packaging rules. This edition does not claim VM validation for those platforms. RHEL security update documentation.

Validation and edition notes

The companion lab record names the image digest, architecture, package versions, running kernel before and after maintenance, exact scripts and observed assertions. It separates successful lab checks from production checks that need your infrastructure. The reusable assessment prompt evaluates the same 28 controls across seven areas and requires evidence for every conclusion.

Treat this as a maintained baseline: review it when the operating-system support window, OpenSSH behavior, package/update mechanism or firewall backend changes. Re-run the lab after editing commands. Retain dated test results without turning those package versions into permanent recommendations.

Operating ownership matrix

SSH access
Accountable role: Identity owner
Executor: Platform engineer
Starting cadence: After access changes; monthly access review

Patch management
Accountable role: Platform owner
Executor: Platform engineer; security triage analyst
Starting cadence: Daily update/failure review; on every relevant advisory

Kernel lifecycle
Accountable role: Platform owner
Executor: Platform engineer; application verifier
Starting cadence: After kernel change/reboot; monthly support review

Firewall exposure
Accountable role: Network owner
Executor: Network engineer; service engineer
Starting cadence: After network changes; daily exposure drift monitoring

Audit and logging
Accountable role: Security operations owner
Executor: Observability engineer; on-call responder
Starting cadence: Continuous pipeline monitoring; monthly delivery exercise

Configuration drift
Accountable role: Configuration owner
Executor: Configuration engineer; service engineer
Starting cadence: After changes; daily comparison

Production verification
Accountable role: Service owner
Executor: Platform engineer; application verifier
Starting cadence: Every rollout and reboot; periodic recovery exercise

Complete the policy template with actual owners, deadlines, acceptance thresholds and escalation paths. The cadences are starting examples; incomplete fields remain unresolved decisions.

Fictional completed change: customer API canary

Illustration only. This is not a production execution receipt, an approved policy or a vendor/CVE assertion. Names, times and thresholds below demonstrate how to complete the worksheet. Replace every value with approved service-specific decisions.

- Target: `api-canary-01`, one of ten API nodes; change `EXAMPLE-1042`; UTC window 2026-11-10 02:00-03:00.
- Accountable service owner and rollout decision maker: Maya Chen. Platform executor: Alex Ionescu. Security triage/exception approver: Sam Patel. Network executor: Noor Ali. Detection executor: Jo Silva. These are fictional roles/people.
- Scope: a vendor-confirmed applicable package fix, its required restart, SSH configuration verification and logging acceptance. Attach the actual advisory, full installed/fixed/active vendor revisions and reviewed transaction before use; this example invents no CVE version.
- Example policy: actively exploited reachable exposure enters incident triage immediately, containment target 4 hours, remediation target 24 hours; other critical fixes 72 hours, high 7 days. Metadata no older than 24 hours. These values are illustrative, not universal recommendations.
- Recovery: independent console exercised in staging; previous approved image and application restore receipt attached. Example recovery objective 15 minutes and no acknowledged transaction loss. No package removal or data migration is approved in this change.
- Traffic: drain only the canary. Require nine healthy serving nodes. Allow management from the approved VPN ranges and application traffic from the approved load-balancer sources. The completed exposure CSV contains the actual addresses and IPv4/IPv6 probe receipts.
- Acceptance: fresh role-appropriate administration works; approved and forbidden network paths behave as recorded; intended fixed code is active; audit/syslog markers arrive within 60 seconds; the routed test alert is acknowledged within 5 minutes.
- Workload: place and retrieve a designated test order, execute its background fulfilment job and check the database/dependency result. Baseline p95 180 ms and error rate 0.2%; for a 15-minute canary window require p95 at most 198 ms and errors at most 0.3%. This example uses a 10% relative latency rise and 0.1 percentage-point error rise, explicitly distinguished.
- Stop triggers: any correctness failure, missing logs/access, unexpected exposure, inadequate healthy capacity or breached threshold. Hold expansion immediately; Maya decides whether to restore the tested image/application state. Re-triage any vulnerability reintroduced by rollback.

Illustrative evidence ledger: at 02:12 the canary health endpoint returns HTTP 200 but the test order cannot be retrieved. The correctness gate fails even though latency is 175 ms. At 02:13 Maya records HOLD and approves the pre-reviewed rollback. At 02:20 the previous state passes order creation/retrieval, the background job and log delivery. Remaining nine nodes are unchanged. At 02:28 Maya records ROLLBACK VERIFIED; Alex owns investigation and a new staging run by 12:00 UTC. The vulnerability remains open under Sam's triage; no exception or PASS is invented.

Resource estimate for this fictional pilot: platform 4 engineer-hours, application verification 2 hours, network and detection 1 hour each, plus the approved maintenance/on-call coverage. Re-estimate from observed pilot effort before proposing the remaining batches of two, three and four nodes. No later batch advances until the service owner records acceptance.

Fixed assessment catalogue LH-1.2

Emit exactly these 28 IDs. PASS requires every criterion. Missing evidence is MANUAL; collection failure is ERROR. Do not convert an exception into a PASS or improve coverage by omitting a check.

SSH-01 / Effective authentication

PASS requires: Root login is disabled; every user/source Match context enforces the approved key/certificate or MFA chain; no unintended password-only route.

Evidence: sshd -t; sshd -T -C for each approved context and root; approved authentication policy.

Applicability: Always applicable to a server offering SSH.

SSH-02 / Identity and key lifecycle

PASS requires: Every privileged identity/key/CA has an owner, justified privilege, revocation path and current access review.

Evidence: Redacted identity inventory, sudo scope, key/CA lifecycle and offboarding evidence.

Applicability: Always applicable; non-SSH administration uses equivalent identity evidence.

SSH-03 / Fresh access verification

PASS requires: A new non-multiplexed session and required sudo action succeed after change; forbidden authentication is rejected in an authorized test.

Evidence: Timestamped new-session receipt, effective forbidden-authentication policy and bounded negative test.

Applicability: Always applicable when SSH is the management path.

SSH-04 / Recovery and revocation

PASS requires: The approved console/break-glass route and an access-revocation exercise meet the organization recovery objective.

Evidence: Dated recovery/revocation exercise, responsible owner, credential storage and retrieval record.

Applicability: Always applicable; do not perform a recovery drill during observational assessment.

PATCH-01 / Supported sources and coverage

PASS requires: OS release, kernel track and repositories are supported, signatures verified and every software channel has an update owner.

Evidence: Release/support evidence, repository signing configuration, entitlement status, third-party/runtime inventory.

Applicability: Always applicable.

PATCH-02 / Fresh update inventory

PASS requires: All required indexes have a successful refresh within policy; all pending/held packages are identified using both ordinary and new-dependency simulations.

Evidence: Refresh receipt for every source, metadata age, apt holds, simulations, dpkg audit.

Applicability: Always applicable; use vendor equivalents outside Ubuntu.

PATCH-03 / Patch execution evidence

PASS requires: The approved pipeline or unattended policy has successful runs, intended origins, monitored failures and controlled restart/reboot behavior.

Evidence: Effective allowed origins and periodic settings, timer/job state, successful transaction and failure alert receipt.

Applicability: Always applicable; a managed pipeline is a valid alternative to unattended installs.

PATCH-04 / Remediation deadlines

PASS requires: Every applicable overdue fix has an approved, unexpired exception; no unowned exposed vulnerability remains.

Evidence: CVE/package mapping, exposure, owner, due date, exception approver and compensating control.

Applicability: Always applicable; no open findings still requires the current inventory and policy.

KERN-01 / Kernel track and boot capacity

PASS requires: The approved supported flavour/meta-package is present; boot/root free space supports the reviewed transaction and recovery image.

Evidence: Package/candidate records, vendor support track, bootloader type, disk capacity.

Applicability: Always applicable to hosts; a container points to an identified host owner, not an automatic N/A.

KERN-02 / Installed and active fix state

PASS requires: Running kernel, installed fixed revision and actual next-boot selection are reconciled; livepatch claims name specific coverage.

Evidence: uname -r; package ownership/version; actual bootloader entries; vendor advisory; livepatch evidence if used.

Applicability: Always applicable to the assessed host. Livepatch itself can be unused without making the control N/A.

KERN-03 / Verified reboot activation

PASS requires: Required reboot is completed within policy and the intended kernel is active with working drivers, networking and workloads.

Evidence: Changed boot ID, post-boot kernel, restart-needs check and workload acceptance record.

Applicability: For a maintenance assessment. For a baseline audit, use a recent approved exercise; absent evidence is MANUAL.

KERN-04 / Fallback boot recovery

PASS requires: A retained recovery option has been exercised and can restore service within the approved recovery objective.

Evidence: Actual fallback/recovery exercise, console access, application recovery and vulnerability re-triage record.

Applicability: Always applicable; an image-replacement strategy can substitute for an older-kernel boot.

NET-01 / Approved exposure inventory

PASS requires: Every listener maps to an owned service with expected protocol, family, source and destination; unused services are removed or explicitly excepted.

Evidence: Service-exposure inventory and ss listener comparison.

Applicability: Always applicable, including non-public hosts.

NET-02 / Effective host and platform policy

PASS requires: Host rules, cloud/network policies and container forwarding permit only the approved paths across IPv4 and every deployed IPv6 path.

Evidence: Effective rules, management-source restrictions, cloud/load-balancer evidence and container mappings.

Applicability: Always applicable. Undeployed layers need an evidence-backed reason, not guessed absence.

NET-03 / Allowed and denied path proof

PASS requires: Authorized probes confirm expected access and rejection from named management and untrusted vantage points, including deployed UDP services.

Evidence: Source/destination/protocol/family matrix with live-listener positive and negative results.

Applicability: Always applicable; no external vantage point means MANUAL for external exposure.

NET-04 / Firewall recovery

PASS requires: The prior approved rules can be restored through a tested independent access path and required services recover.

Evidence: Rollback exercise, saved exact policy and post-restore access/health evidence.

Applicability: Always applicable; observational mode consumes existing evidence only.

AUD-01 / Useful event coverage

PASS requires: Persistent audit rules and authentication/privilege logs cover the approved event inventory and controlled events are retrievable.

Evidence: Loaded and persistent rules, event inventory, redacted audit/authentication event examples.

Applicability: Always applicable; alternative audit mechanisms require documented equivalent evidence.

AUD-02 / Retention and pressure behavior

PASS requires: Journal and audit-log limits separately match policy; rotation, free-space/queue alerts and failure actions are verified under safe test conditions.

Evidence: Effective journald and auditd settings, retention calculation, rotation result, disk/queue alert exercise.

Applicability: Always applicable; a configured limit alone is not a retention guarantee.

AUD-03 / Authenticated central delivery

PASS requires: Required event streams reach an authenticated central receiver; a failed certificate is rejected and a bounded receiver outage is detected and recovered.

Evidence: TLS peer verification, unique marker receipt, audit-event receipt, negative TLS result, outage/recovery and queue evidence.

Applicability: Always applicable for this production baseline; local-only logging is an exception, not N/A.

AUD-04 / Actionable detection

PASS requires: Synchronized timestamps, central access/retention controls and a routed alert with human acknowledgement meet policy.

Evidence: Clock status, receiver retention/access policy, alert receipt/acknowledgement and response owner.

Applicability: Always applicable; a lab sink is not proof of production alert handling.

DRIFT-01 / Trusted desired state

PASS requires: The approved baseline, scope and revision are stored independently of the host and tied to an authorized change.

Evidence: Reviewed repository/artifact revision, external manifest digest, scope and approver.

Applicability: Always applicable; a root-writable local manifest alone cannot pass.

DRIFT-02 / Complete file inventory

PASS requires: Approved paths match names, types, owners, permissions, symlink targets and file content; extra/missing files are detected.

Evidence: Inventory comparison plus positive/negative regression evidence.

Applicability: Always applicable; explicit exclusions must have owners and reasons.

DRIFT-03 / Runtime and package drift

PASS requires: Effective SSH policy, active firewall, enabled services, installed packages and kernel track match the approved inventory.

Evidence: Versioned runtime/package/service inventories and independent comparisons.

Applicability: Always applicable; filesystem hashes alone cannot pass.

DRIFT-04 / Safe baseline promotion

PASS requires: Missing/unreadable/unsupported input prevents baseline replacement; reconciliation preserves the approved record and exceptions expire.

Evidence: Atomic promotion negative test, restoration test, approval record, drift ticket/exception expiry.

Applicability: Always applicable.

VERIFY-01 / Defined acceptance and recovery

PASS requires: Named owner has approved success thresholds, observation window, rollback trigger, recovery objective and maintenance dependencies before change.

Evidence: Completed change worksheet and recent application recovery evidence.

Applicability: Always applicable.

VERIFY-02 / OS and access acceptance

PASS requires: Fresh admin access, intended kernel, completed package state, expected listeners, healthy critical units and logging pass after change/reboot.

Evidence: Timestamped pre/post checks and unresolved-error list.

Applicability: Always applicable; known pre-existing failures require an approved exception.

VERIFY-03 / Workload acceptance

PASS requires: A representative user transaction, background job and dependency path meet agreed correctness/latency/error thresholds during the observation window.

Evidence: Actual transaction/job receipts and monitoring comparison to the recorded baseline.

Applicability: Always applicable; a synthetic lab application does not certify the real workload.

VERIFY-04 / Controlled rollout decision

PASS requires: Canary acceptance, rollback rehearsal, staged rollout and final owner sign-off are recorded; failed gates stop advancement.

Evidence: Release decision record with canary count, gates, rollback result and sign-off.

Applicability: Always applicable; assessment alone never authorizes a production rollout.

Lab validation record

Tested on 3–4 November 2026 in a real Ubuntu 24.04.5 LTS ARM64 VM through Lima 2.2.1 and Apple Virtualization Framework: 2 CPUs, 3 GiB RAM, 16 GiB disk, no host directory mounts. The active kernel moved from 6.8.0-142 to 6.8.0-147; a one-time fallback boot exercised 142 before returning to 147. Results below describe lab mechanisms, not complete production-control outcomes.

SSH - PASS · LAB
Fresh pinned SSH identity and exact approved privilege check passed; root/password-only paths rejected. Separate password-required and restricted automation roles behaved correctly and were removed.

Patching - PASS · LAB
70 packages upgraded, followed by 4 kernel meta-packages and 6 dependencies. Both final simulations, timers and reboot policy were checked; a later strict index refresh and TLS module install succeeded.

Kernel lifecycle - PASS · LAB
Activated 6.8.0-147, booted the retained 6.8.0-142 once through GRUB, then returned to 147 with distinct boot IDs and no failed units.

Firewall exposure - PASS · LAB
Independent namespaces proved allowed/denied SSH and live TCP 443/8080 filtering over IPv4/IPv6. Exact saved firewall files were restored.

Audit logging - PASS · LAB
Audit capture/rotation retained prior evidence. Mutual TLS, audit/syslog inputs and wrong-peer rejection passed. All 300 queued sequence IDs reconciled with no gaps/duplicates; real queue pressure reached a local fixture alert sink.

Configuration drift - PASS · LAB
Expanded 185-entry inventory matched. Added APT, repository, journald, UFW, sudo and rsyslog overrides were detected; private report capture and collector status classification passed.

Production verification - PASS · FIXTURE
OS/access checks passed after recovery. A synthetic transaction detected a wrong result despite HTTP 200, held advancement and passed after rollback. Actual service acceptance remains open.

One Ubuntu ARM64 VM is not a fleet certification. Cloud/internet/container/UDP paths, PAM MFA, livepatch, production load, disk-full behavior and external alert routing, central retention and human acknowledgement, independent baseline trust, identity governance, backup/data recovery and real production transactions require separate evidence. Lab TCP/TLS recovery is not an exactly-once guarantee. The worked assessment keeps these production requirements unresolved against all 28 fixed controls.

Get the full prompt, toolkit and editable sources from the GitHub release assets. Collect on the server; analyse the redacted report on a separate workstation. Re-run the lab after changing executable guidance.